Phần 1. Phân tích tư duy và so sánh kỹ thuật

1. Hai sai lầm cốt lõi trong tư duy của Nam

- **Sai lầm 1: Đồng nhất tài liệu mã nguồn dự án với văn bản in ấn văn phòng (Rich Text vs. Plain Text)**

Nam cho rằng cần font chữ và màu sắc trực quan mới là chuyên nghiệp. Trong quản lý dự án phần mềm, sự chuyên nghiệp thể hiện ở tính chuẩn hóa, khả năng đọc nhanh trên mọi môi trường (terminal, trình duyệt web GitHub, VS Code) mà không cần cài thêm bộ phần mềm Microsoft Office nặng nề. File Word (`.docx`) không thể hiển thị trực quan trực tiếp trên GitHub Web, buộc các thành viên phải tải về máy mới đọc được.

- **Sai lầm 2: Bỏ qua cơ chế theo dõi phiên bản dòng-theo-dòng (Line-by-line Diff) của Git**

File `.docx` thực chất là một gói lưu trữ nén nhị phân (ZIP chứa các file XML, quan hệ, hình ảnh). Git coi `.docx` là file nhị phân (binary). Khi Nam chỉ sửa một dấu phẩy, toàn bộ mã hash nhị phân của khối nén thay đổi hoàn toàn. Git không thể nhận biết dòng nào thay đổi để gộp (auto-merge), dẫn đến việc báo conflict toàn bộ file và làm phình to lịch sử commit của repo.

2. Vì sao Markdown nhẹ hơn, an toàn hơn và chống hư hại file

Tiêu chí	Microsoft Word (.docx)	Markdown (.md)
Bản chất dữ liệu	Gói nhị phân nén (Binary / ZIP + XML)	Văn bản thô thuần túy (Plain Text UTF-8)
Dung lượng lưu trữ	Thường từ vài chục KB đến hàng chục MB (chứa metadata, style, schema)	Rất nhẹ, chỉ vài byte đến vài KB (chỉ lưu các ký tự thực tế)
Hỗ trợ Git Diff & Merge	Không hỗ trợ; Git xử lý dạng binary $\rightarrow$ báo conflict toàn tệp	Phân tích từng dòng (+/-); tự động merge chuẩn xác khi nhiều người sửa khác dòng
Chống hư hại file (Data Corruption)	**Rất yếu:** Mất chỉ 1 byte cấu trúc nén khiến toàn bộ tệp bị lỗi giải nén (corrupted) và không mở được	**Tuyệt đối an toàn:** Nếu mất hoặc sai vài byte, chỉ chữ tại vị trí đó bị lỗi; 99% nội dung còn lại vẫn đọc được bình thường